

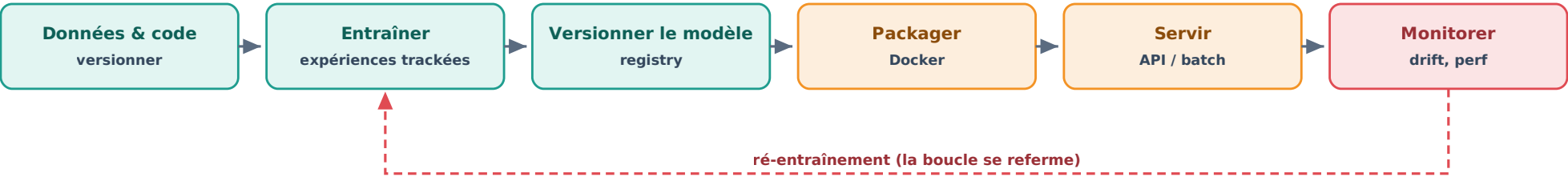
Du notebook à la prod — le cycle MLOps

Industrialiser un modèle : reproductible, déployé, surveillé · Marianne A. · relie M1B2 (packager/servir)

L'idée. Un modèle qui ne tourne qu'en *notebook* n'a aucune valeur tant qu'il n'est pas **reproductible**, **déployé** et **surveillé**. Le **MLOps** industrialise ce passage : il transforme un script exploratoire en un service fiable, versionné et suivi dans le temps.

Repère ~15 % des modèles atteignent réellement la prod — le goulot d'étranglement n'est pas le modèle, c'est *tout ce qu'il y a autour* (données, tests, déploiement, monitoring).

1 Le cycle MLOps



2 Étape par étape

ÉTAPE	OUTILS TYPIQUES	LIVRABLE
Versionner données & code	Git, DVC	reproductibilité.
Tracker les expériences	MLflow	métriques comparables, historique.
Versionner le modèle	Model Registry (MLflow), joblib	artefact + <i>model card</i> .
Packager	Docker	image reproductible (mêmes dépendances partout).

ÉTAPE	OUTILS TYPIQUES	LIVRABLE
Servir	FastAPI / BentoML (temps réel) ou job batch	endpoint / prédictions.
Orchestrer	Airflow / Kubeflow	pipelines planifiés et reproductibles.
Monitorer	Prometheus / Grafana, détection de drift	alertes, tableaux de bord.

3 Ce qui distingue la prod du notebook

ENJEU	EN NOTEBOOK	EN PROD
Reproductibilité	« ça marche chez moi »	seeds fixés, versions épinglées, Docker.
Tests	aucun	tests de données et de code, CI/CD.
Déploiement	manuel	automatisé, avec rollback possible.
Données	figées	elles dérivent dans le temps (drift).
Suivi	on regarde une fois	monitoring continu de la performance.

4 Le drift, ennemi n°1 en prod

En prod, les données ne restent pas figées comme dans un notebook : elles **dérivent**. On distingue deux dérives. Le **data drift** : la *distribution des entrées* change (nouveaux profils d'utilisateurs, saisonnalité, capteur recalibré) alors que la relation entrées→cible reste la même. Le **concept drift** : c'est la *relation entrées→cible elle-même* qui change (les habitudes d'achat évoluent, une fraude change de mode opératoire), même si les entrées se ressemblent.

Dans les deux cas, la **performance se dégrade silencieusement** : le modèle continue de répondre, sans erreur visible, mais ses prédictions deviennent de moins en moins justes. D'où la nécessité de **monitorer** en continu (distributions, métriques) pour détecter la dérive tôt et **déclencher un ré-entraînement** sur des données fraîches.

C'est précisément pour ça que le cycle boucle : Monitorer renvoie vers Entraîner.

La règle d'or. Commence **simple** — un pipeline joblib + une API + un minimum de tracking et de tests — *avant* d'empiler Kubernetes. Monte en industrialisation seulement quand le besoin est là. (Pour la GenAI, l'équivalent est le **LLMOps**.)

À retenir. Le MLOps fait passer un modèle du notebook à la prod par un cycle qui boucle — versionner, entraîner, packager, servir, monitorer — et ce sont la reproductibilité, les tests, le déploiement automatisé et la surveillance du drift, bien plus que le modèle lui-même, qui décident s'il atteint vraiment la production.